

sa 종합 관찰 프로브

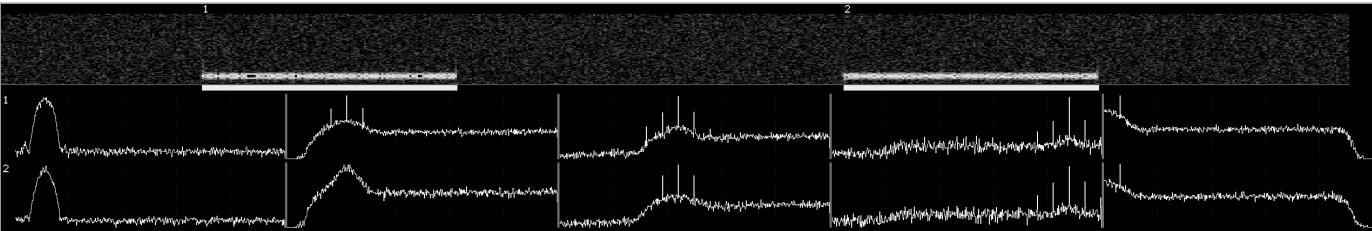
스펙트로그램 + find_bursts 검출 + 버스트별 변조 판독($x^2 \cdot x^4 \cdot x^8 \cdot AM$)을 실행 한 번, 그림 한 장으로

코드 172줄 · 총 4쪽 | 2026-08-22 · 4cc441b+수정중 | strip.py 를 대체한다 — strip 을 아직 안 쳤다면 이것만 치면 된다. 출력은 PNG 파일.

- ① 이 코드를 **signus/** 옆에 **sa.py** 로 저장한다.
- ② **python3 sa.py kat** — 아래 기대 출력과 글자까지 비교하고 **sa-kat.png** 를 열어 기대 그림과 비교한다 (다르면 필사 오타).
- ③ **python3 sa.py <캡처파일> [K]** — **<캡처파일>.sa.png** 생성 (에너지 상위 K개 버스트 판독, 기본 4). 그림 위 = 버스트 번호 붙은 스펙트로그램 + 검출 띠(흰색=검출, 빈 띠=통짜 미검출), 아래 = 버스트별 [PSD | x^2 | x^4 | x^8 | AM] 다섯 판.
- ④ 회신: 버스트 표 몇 줄 + **sa cap ...** 요약 줄(#코드까지) + 판독 행에서 바늘이 선 판 이름 한 마디.

바늘이 처음 서는 판	판정	바늘 위치	AM 봉우리
x^2	BPSK	2·fc (mod fs, 접힌 축)	심볼레이트
x^4	QPSK	4·fc	
x^8	8PSK	8·fc	
없음	PSK 아님 (잡음꼴·FSK 등)	—	—

```
n 80000 fs 10000 길이 8.000s 포락선 분리(중앙 기준) 1.18 decades
find_bursts → 2개
  1 샘플 [ 11904, 27072) 1516.8 ms 전력 +4.0 dB ★판독
  2 샘플 [ 49984, 65024) 1504.0 ms 전력 +4.0 dB ★판독
sa b1 p2 f1100 d47 p4 f2200 d42 p8 f4400 d33 am f294 d39 #mmy9
sa b2 p2 f1123 d31 p4 f2200 d40 p8 f4400 d31 am f294 d38 #0fj5
sa kat n80000 f10000 s8.0 fb2 ev118 #7n3t
sa-kat.png 저장 — 위: 스펙트로그램+검출 띠+버스트 번호, 아래: ★버스트별 [PSD |  $x^2$  |  $x^4$  |  $x^8$  | AM] ( $x^2$  바늘=BPSK,  $x^4$ =QPSK,  $x^8$ =8PSK, AM 봉우리=심볼
```



기대 그림: 위 스트립에 버스트 1(BPSK)·2(QPSK)와 검출 띠, 아래 판독 2행 — 1행은 x^2 판에 바늘, 2행은 x^2 은 언덕뿐이고 x^4 판에 바늘. 한 줄이 100칸을 넘으면 **↳** 이 어짐 — 붙여서 입력.

```

1  """sa.py - 종합 관찰 프로브 (장비 필사용, signus/ 옆에 저장). 한 번에:
2  디버그 출력(포락선·버스트 표) + <캡처>.sa.png (스펙트로그램 + 검출 띠 + 버스트 번호 +
3  상위 버스트별 [PSD |  $x^2$  |  $x^4$  |  $x^8$  | AM검파] 판독 행 - 바늘이 처음 서는 판이 변조).
4      python3 sa.py kat          # 자기검증: 표지 기대 줄과 비교, sa-kat.png 생성
5      python3 sa.py <캡처파일> [K] # 판독할 버스트 수 K (기본 4, 에너지 상위부터)
6   $x^m$  판은 접힌 축( $0 \sim fs/2$ , 음수 주파수를 겹쳐 최대값) - 바늘 위치가  $m \cdot fc \pmod{fs}$  다.
7  """
8  import struct
9  import sys
10 import zlib
11
12 import numpy as np
13 from scipy.signal import stft, welch
14
15 from signus import dsp, sigio
16 from signus.cli import check_code
17
18 GLYPH = {"0": (14, 17, 19, 21, 25, 17, 14), "1": (4, 12, 4, 4, 4, 4, 14),
19          "2": (14, 17, 1, 2, 4, 8, 31), "3": (31, 2, 4, 2, 1, 17, 14),
20          "4": (2, 6, 10, 18, 31, 2, 2), "5": (31, 16, 30, 1, 1, 17, 14),
21          "6": (6, 8, 16, 30, 17, 17, 14), "7": (31, 1, 2, 4, 8, 8, 8),
22          "8": (14, 17, 17, 14, 17, 17, 14), "9": (14, 17, 17, 15, 1, 2, 12)}
23
24
25 def draw_text(img, row, col, text, s=2):
26     for ch in text:
27         for r, bits in enumerate(GLYPH[ch]):
28             for c in range(5):
29                 if bits >> (4 - c) & 1:
30                     img[row + r * s:row + r * s + s, col + c * s:col + c * s + s] = 1.0
31             col += 6 * s
32
33
34 def png_gray(img, path):
35     img = np.clip(img * 255, 0, 255).astype(np.uint8)
36     h, w = img.shape
37
38     def chunk(tag, data):
39         c = tag + data
40         return struct.pack(">I", len(data)) + c + struct.pack(">I", zlib.crc32(c))
41
42     raw = b"".join(b"\x00" + img[r].tobytes() for r in range(h))
43     with open(path, "wb") as fh:
44         fh.write(b"\x89PNG\r\n\x1a\n"
45                + chunk(b"IHDR", struct.pack(">IIBBBBB", w, h, 8, 0, 0, 0, 0))
46                + chunk(b>IDAT", zlib.compress(raw, 6)) + chunk(b"IEND", b""))
47
48
49 def panel(fvals, dbvals, wp=500, hp=120, grid=()):
50     """스펙트럼 곡선 한 판 - 바늘 보존을 위해 표시 칸마다 최대값 풀림."""
51     cols = np.clip((fvals / fvals.max() * (wp - 1)).astype(int), 0, wp - 1)
52     cur = np.full(wp, -1e9)
53     np.maximum.at(cur, cols, dbvals)
54     bad = cur < -1e8
55     cur[bad] = np.interp(np.flatnonzero(bad), np.flatnonzero(~bad), cur[~bad])
56     g_lo, g_hi = np.percentile(cur, 5) - 2, cur.max() + 2
57     img = np.zeros((hp, wp))
58     for gf in grid:
59         img[:, 6, int(gf / fvals.max() * (wp - 1))] = 0.35
60     rows = ((hp - 1) * (1 - np.clip((cur - g_lo) / (g_hi - g_lo), 0, 1))).astype(int)

```

```

61     for cx in range(wp):
62         r0, r1 = (rows[cx], rows[cx - 1]) if cx else (rows[0], rows[0])
63         img[min(r0, r1):max(r0, r1) + 1, cx] = 1.0
64     return img
65
66
67 def peak(fv, dbv, fmin=20.0):
68     """판 하나의 최대 바늘: 주파수(Hz)와 중앙값 대비 돌출(dB) -- 숫자로 받아칠 수 있게."""
69     ok = fv >= fmin
70     i = int(np.argmax(dbv[ok]))
71     return f"f{round(float(fv[ok][i]))} d{round(float(dbv[ok][i] - np.median(dbv[ok])))}"
72
73
74 def burst_row(z, fs, label):
75     """한 버스트의 판독 행: [PSD | x^2 | x^4 | x^8 | AM]. x^m 은 음수 주파수를 접는다."""
76     grid = tuple(fs / 2 * k / 5 for k in (1, 2, 3, 4))
77     fr, praw = welch(z.real, fs=fs, nperseg=min(2048, z.size))
78     ps = [panel(fr, 10 * np.log10(praw + 1e-18), grid=grid)]
79     nfft, nums = 1 << 15, []
80     f = np.fft.fftfreq(nfft, 1 / fs)
81     half = nfft // 2
82     for m in (2, 4, 8):
83         mag = np.abs(np.fft.fft((z ** m) * np.hanning(z.size), nfft))
84         fold = np.maximum(mag[:half], np.r_[mag[0], mag[1:half][::-1] * 0
85                                             + mag[half + 1:][::-1]])
86         ps.append(panel(f[:half], 20 * np.log10(fold + 1e-12), grid=grid))
87         nums.append(f"p{m} " + peak(f[:half], 20 * np.log10(fold + 1e-12)))
88     u = np.abs(z) ** 2
89     u = u - u.mean()
90     su = np.abs(np.fft.rfft(u * np.hanning(u.size), nfft))
91     ps.append(panel(np.fft.rfftfreq(nfft, 1 / fs), 20 * np.log10(su + 1e-12), grid=grid))
92     nums.append("am " + peak(np.fft.rfftfreq(nfft, 1 / fs), 20 * np.log10(su + 1e-12)))
93     div = np.full((120, 4), 0.5)
94     row = np.hstack(sum([p, div] for p in ps[:-1]), []) + [ps[-1]]
95     tag = np.zeros((120, 26))
96     draw_text(tag, 4, 4, label)
97     return np.hstack([tag, row]), " ".join(nums)
98
99
100 if len(sys.argv) < 2:
101     raise SystemExit("사용법: python3 sa.py kat | python3 sa.py <캡처파일> [버스트수 K]")
102 arg = sys.argv[1]
103 kmax = int(sys.argv[2]) if len(sys.argv) > 2 else 4
104 if arg == "kat":
105     fs, n = 10000.0, 80000
106     rng = np.random.default_rng(21) # 시드 고정 -- numpy 가 재현을 보장한다
107     x0 = 0.18 * rng.standard_normal(n)
108     t = np.arange(15000) / fs
109     for s0, fc, ph in ((12000, 550.0, 2), (50000, 550.0, 4)): # 앞=BPSK, 뒤=QPSK
110         up = np.zeros(15000, complex)
111         up[:34] = np.exp(2j * np.pi * rng.integers(0, ph, 442) / ph)
112         sym = np.convolve(up, np.hanning(68), "same") # 간이 펄스 성형
113         sym /= np.sqrt(np.mean(np.abs(sym) ** 2))
114         x0[s0:s0 + 15000] += (sym * np.exp(2j * np.pi * fc * t)).real * np.sqrt(2)
115     name = "sa-kat"
116 else:
117     x0, meta = sigio.read(arg)
118     fs = meta.fs
119     name = arg + ".sa"
120 xa = dsp.analytic(x0)

```

```

121 xa = xa - xa.mean()
122 n = xa.size
123 fb = dsp.find_bursts(xa, fs)
124 full = fb == [(0, n)]
125 pw = np.abs(xa) ** 2
126 lp = np.log10(np.maximum(np.convolve(pw, np.ones(64) / 64, "same"), 0) + 1e-20)
127 et = np.percentile(lp, 50)
128 ev = float(lp[lp >= et].mean() - lp[lp < et].mean())
129 print(f"n {n} fs {fs:.0f} 길이 {n / fs:.3f}s 포락선 분리(중앙 기준) {ev:.2f} decades")
130 print(f"find_bursts → {'통짜 [(0,n)] = 미검출' if full else str(len(fb)) + '개'}")
131 order = sorted(range(len(fb)), key=lambda i: -float(pw[fb[i][0]:fb[i][1]].sum()))
132 pick = sorted(order[:kmax]) if not full else []
133 for i, (s0, e0) in enumerate(fb if not full else []):
134     star = " ★판독" if i in pick else ""
135     print(f" {i + 1:>2} 샘플 [{s0:>8}, {e0:>8}] {1000 * (e0 - s0) / fs:8.1f} ms"
136           f" 전력 {10 * np.log10(pw[s0:e0].mean() / (pw.mean() + 1e-30) + 1e-30):+5.1f} dB{star}")
137
138 x_disp = x0.real if np.iscomplexobj(x0) else x0
139 _, _, z = stft(x_disp, fs=fs, window="hamming", nperseg=256, noverlap=128,
140               return_onesided=True, boundary=None, padded=False)
141 db = 10 * np.log10(np.abs(z) ** 2 + 1e-12)
142 lo = float(np.percentile(db, 25))
143 rg = float(max(10.0, min(35.0, np.percentile(db, 99.5) - lo)))
144 spec = np.clip((db - lo) / rg, 0, 1)[::-1]
145 kp = max(1, spec.shape[1] // 4000)
146 spec = spec[:, :spec.shape[1] // kp * kp].reshape(spec.shape[0], -1, kp).max(2)
147 pw_row = 26 + 5 * 500 + 4 * 4 # 판독 행 폭 -- 스트립을 여기에 맞춰 늘린다
148 fx = max(1, pw_row // spec.shape[1])
149 spec = np.repeat(spec, fx, axis=1)
150 ncol = spec.shape[1]
151 mark = np.zeros((18, ncol))
152 lane = np.zeros(ncol)
153 for i, (s0, e0) in enumerate(fb if not full else []):
154     c0 = (s0 // 128) // kp * fx
155     c1 = min(ncol - 1, (e0 // 128) // kp * fx)
156     lane[c0:c1 + 1] = 1.0
157     draw_text(mark, 2, min(c0 + 2, ncol - 14 * len(str(i + 1))), str(i + 1))
158 rows = [mark, spec, np.full((2, ncol), 0.35), np.tile(lane, (10, 1)),
159         np.full((6, ncol), 0.0)]
160 width = max(ncol, pw_row)
161 rows = [np.pad(r, ((0, 0), (0, width - r.shape[1]))) for r in rows]
162 for i in pick:
163     r, num = burst_row(xa[fb[i][0]:fb[i][1]], fs, str(i + 1))
164     ln = f"sa b{i + 1} {num}" # 바늘 주파수·돌출 -- 받아칠 판독 숫자줄
165     print(f"{ln} #{check_code(ln)}")
166     rows += [np.pad(r, ((0, 0), (0, width - r.shape[1]))), np.full((6, width), 0.0)]
167 png_gray(np.vstack(rows), name + ".png")
168 line = (f"sa {'kat' if arg == 'kat' else 'cap'} n{n} f{fs:.0f} s{n / fs:.1f}"
169         f" fb{0 if full else len(fb)} ev{round(100 * ev)}")
170 print(f"{line} #{check_code(line)}")
171 print(f"{name}.png 저장 - 위: 스펙트로그램+검출 띠+버스트 번호, 아래: ★버스트별"
172       " [PSD|x2|x4|x8|AM] (x2 바늘=BPSK, x4=QPSK, x8=8PSK, AM 봉우리는심볼레이트)")

```